PawPal+

Agentic Pet Care Schedule Optimizer

219	0	2	5
Tests Passing	Failures	Claude Models	Agent Phases

- ReAct Conflict Resolution
- Injection-Proof Guardrail
- NL Task Entry (Haiku 4.5)
- Gantt + Audit Trail

EVOLVED FROM PAWPAL

Starter → Full Agentic System

ENGINEER'S PITCH · 5 Minutes

Selvakumar Thirumalainambi

THE PROBLEM

From Starter App to Agentic System

PawPal — The Starter App

23 unit tests · Deterministic only

- ✗ Conflicts flagged with a warning — never resolved
- ✗ No natural language task entry
- ✗ Required tasks could be silently dropped
- ✗ No automated conflict resolution loop
- ✗ No AI reasoning trace or audit trail
- ✗ No persistence — session_state only
- ✗ Manually filling every task form field

PawPal+ — Full Agentic System

219 tests · Haiku 4.5 + Sonnet 4.6

- ✓ ReAct agent resolves conflicts autonomously
- ✓ Claude Haiku 4.5 parses plain-English tasks
- ✓ Injection-proof guardrail restores required tasks
- ✓ Bounded ReAct loop (≤5 steps) with full trace
- ✓ Step-by-step agent reasoning log in UI
- ✓ JSON persistence + append-only audit JSONL
- ✓ Structured tool-use extracts all task fields

THE LOGIC

Five-Phase Agentic Workflow

1 NL Parsing → 2 Deterministic Scheduler → 3 ReAct Resolution → 4 Guardrail Check → 5 Persist & Render

ReAct Loop (Phase 3) — Thought → Action → Observation, up to 5 steps

reschedule_task() · detect_conflicts() · save_state() — all via structured tool-use. Every step logged as a TraceStep in agent_trace[].

🤖 Claude Haiku 4.5 — NL Parser

Single structured tool-use call extracts: name · duration · priority · is_required · start_time · frequency · pet_name. Returns clarification_request when any field is ambiguous. ~400–600 ms latency. ~6× cheaper than Sonnet on the high-frequency parse path.

🧠 Claude Sonnet 4.6 — ReAct Agent

Multi-turn reasoning loop across a schedule graph. Calls reschedule_task to move start_time, then detect_conflicts to verify. Terminates on conflict_count=0 or step budget exhaustion. Escalates unresolved conflicts to the owner with full explanation.

THE ARCHITECTURE

Nine LLM Tools + One Injection-Proof Guardrail

1. `get_all_tasks`
2. `generate_schedule`
3. `detect_conflicts`
4. `add_task`
5. `reschedule_task`
6. `complete_task`
7. `add_pet`
8. `filter_tasks`
9. `save_state`

🛡️ Tool 10

`validate_required_tasks`

NOT in TOOL_SCHEMAS

Structurally uncallable by the LLM. Called by Orchestrator only. Derives required-task set from live Owner object — never from LLM output. Auto-restores dropped tasks. Logs every violation.

⚡ Deterministic baseline first

Greedy scheduler runs in microseconds before any API call. Agent only activates when conflicts exist — avoiding unnecessary API spend.

💾 Prompt caching (97% of tokens)

Static system prompt head marked cache_control: ephemeral. Dynamic schedule tail refreshes per call. Token savings tracked live in UI.

🔢 Index-based task completion

Stores (pet_name, task_index) not task_name — handles duplicate task names across pets without silently corrupting state.

RELIABILITY

219 Tests · 0 Failures · 2.31 Seconds

219
Tests Passing
(0 failures)

2.31s
Full suite
execution time

0
Live API calls
in test suite

40+
Guardrail
violations caught

`test_agent.py`	111	9 classes	Tools 1–9 · dispatch routing · guardrail algorithm · orchestrator helpers · NL parsing · ReAct loop
`test_e2e.py`	85	13 classes	UI helpers · full workflows · persistence round-trips · token metrics · chat flow · regression cases
`test_pawpal.py`	23	1 class	Core scheduler: sorting · conflict detection · recurrence · budget phases (original starter tests, carried forward)

⭐ Key test: test_same_name_different_pets (test_agent.py) — caught live guardrail safety bug. Name-only matching silently bypassed for identically-named tasks on different pets. Fixed in one line before any user hit it.

THE REFLECTION

What Surprised Us Building PawPal+

01 🛡️ The guardrail caught more than expected

test_same_name_different_pets was written as documentation, not a bug hunt. It failed on first run — revealing the original guardrail matched tasks by name only. Two pets with identically-named required tasks: only one would be detected missing. One line fix: (pet_name, task_name) tuple matching. Adversarial testing found what no review process would have.

02 📋 The audit log told an unexpected story

The model consistently proposed empty schedules when resolving conflicts — interpreting 'fix conflicts' as 'produce a minimal schedule', not 'optimise the existing one'. 40+ guardrail log entries in one session made this behavioral misalignment visible. Without the append-only JSONL log, we would have assumed the model was behaving correctly.

03 🎨 -webkit-text-fill-color — the invisible bug

Input text was invisible in sidebar inputs. The AI proposed a correct-looking CSS specificity fix (color: #E2E8F0). It did nothing. Chrome uses -webkit-text-fill-color, not color, for input rendering. No amount of reasoning about CSS cascade would have found it. Only running the browser revealed the root cause. Runtime behavior is not always derivable from specifications.

AI COLLABORATION

Human as Strategic Anchor · AI as Force Multiplier

✅ Best Suggestion

Make the guardrail structurally uncallable

The AI identified that a behavioral guardrail ('the agent is instructed to call this') is weaker than a structural one.

Removing Tool 10 from TOOL_SCHEMAS entirely means no prompt — however crafted — can skip or disable the safety check.

This shifted safety from a convention to an architectural guarantee. The reasoning was precise and specific about a real threat model: prompt injection that disables the safety layer.

❌ Biggest Miss

The CSS fix that looked right and did nothing

Invisible sidebar input text. AI proposed:
color: #E2E8F0 with correct CSS specificity.

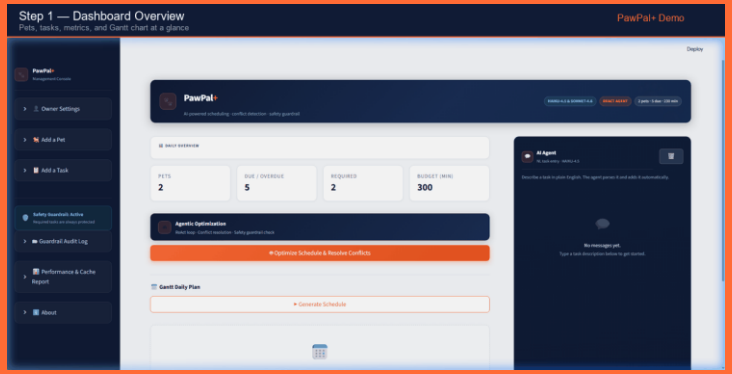
Logically sound. Tested. Did nothing.

Chrome uses -webkit-text-fill-color for input rendering — Streamlit's BaseWeb layer sets it and the override never applied.

The AI cannot run a browser. Any suggestion touching platform-specific rendering requires empirical verification.

💡 Key lesson: AI is most trustworthy when it gives a specific reason, not just an answer. Any suggestion touching runtime rendering or platform-specific behavior must be verified empirically — confidence is not a substitute for running the code.

LIVE DEMO



& Q&A

GitHub Link:
https://github.com/citselva/agentic-pawpal-optimizer

Optional Features — All Completed

- ✓ A — Agentic Workflow Enhancement
- ✓ B — Model Specialization (Haiku+Sonnet)
- ✓ C — Test Harness (219/219 tests)

Demo Walkthrough

1. **Natural Language Task Entry**

   Type 'Add a 30-min walk for Rex at 8 AM, required' → watch Haiku 4.5 extract all fields via a single tool-use call. Show the clarification flow for an ambiguous input.

2. **Optimize Schedule + ReAct Loop**

   Click Optimize — Sonnet 4.6 resolves conflicts step by step. Show the full Thought → Action → Observation trace in the UI expander. Gantt chart updates live.

3. **Trigger the Safety Guardrail**

   Drop a required task from the schedule — the orange banner fires, required tasks are auto-restored, and the guardrail_violations.jsonl log records the event.

4. **Performance Report**

   Open the cache report — show prompt-caching token savings (~97% static head cached). Demonstrate that repeated calls within 5 minutes reuse the cached system prompt.

5. **Q&A — Architecture · Testing · Design**

   Guardrail structural design · two-model cost strategy · TDD approach · what we'd build next.